

PRACTICAL – 6

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main()
{
    char message[100];
    char response[150];

    mkfifo(FIFO1, 0666);
    mkfifo(FIFO2, 0666);

    printf("Server started...\n");

    int read_fd = open(FIFO1, O_RDONLY);
    int write_fd = open(FIFO2, O_WRONLY);

    while (1)
    {
        memset(message, 0, sizeof(message));

        read(read_fd, message, sizeof(message));

        printf("Client message: %s\n", message);
```

```

    if (strcmp(message, "exit") == 0)
    {
        strcpy(response, "Server shutting down.");
        write(write_fd, response, strlen(response) + 1);
        break;
    }

    snprintf(response, sizeof(response),
             "Server processed: %s", message);

    write(write_fd, response, strlen(response) + 1);
}

close(read_fd);
close(write_fd);

unlink(FIFO1);
unlink(FIFO2);

return 0;
}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO1 "client_to_server"
#define FIFO2 "server_to_client"

int main()
{
    char message[100];
    char response[150];

    int write_fd = open(FIFO1, O_WRONLY);
    int read_fd = open(FIFO2, O_RDONLY);

    while (1)
    {

```

```

printf("Enter message: ");
fgets(message, sizeof(message), stdin);

message[strcspn(message, "\n")] = '\0';

write(write_fd, message, strlen(message) + 1);

memset(response, 0, sizeof(response));

read(read_fd, response, sizeof(response));

printf("Server response: %s\n", response);

if (strcmp(message, "exit") == 0)
    break;
}

close(write_fd);
close(read_fd);

return 0;
}

```

The image shows two terminal windows side-by-side. The left window shows the server being compiled and run, while the right window shows the client being run and interacting with the server.

```

lahari@DESKTOP-AIC33F7: ~/OS_Lab$ cd practical6
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ nano server.c
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ nano client.c
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ gcc server.c -o server
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ gcc client.c -o client
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ ./server
Server started...
Client message: Hi
Client message: Ya Not Good
Client message: ok bye
Client message: exit
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$

lahari@DESKTOP-AIC33F7: ~/OS_Lab$ cd practical6
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$ ./client
Enter message: Hi
Server response: Server processed: Hi
Enter message: Ya Not Good
Server response: Server processed: Ya Not Good
Enter message: ok bye
Server response: Server processed: ok bye
Enter message: exit
Server response: Server shutting down.
lahari@DESKTOP-AIC33F7: ~/OS_Lab/practical6$

```

Signal.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

```

```

void handle_signal(int sig)
{
    if (sig == SIGINT)
    {
        printf("\nSIGINT received. Ctrl+C detected.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received. Terminating process...\n");
        exit(0);
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received. User-defined event occurred.\n");
    }
}

int main()
{
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    signal(SIGUSR1, handle_signal);

    printf("Process ID: %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(2);
    }
    return 0;
}

```

```
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ nano signal.c
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ gcc signal.c -o signal
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ ./signal
Process ID: 590
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
```

```
lahari@DESKTOP-AIC33F7:~$ cd OS_Lab
lahari@DESKTOP-AIC33F7:~/OS_Lab$ cd practical6
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ ./client
Enter message: Hi
Server response: Server processed: Hi
Enter message: Ya Not Good
Server response: Server processed: Ya Not Good
Enter message: ok bye
Server response: Server processed: ok bye
Enter message: exit
Server response: Server shutting down.
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ kill -USR590
-bash: kill: USR590: invalid signal specification
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ kill -USR1 590
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ kill -TERM 590
lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6$ |
```

SIGUSR1 received. User-defined event occurred.

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

Program is running...

SIGTERM received. Terminating process...

lahari@DESKTOP-AIC33F7:~/OS_Lab/practical6\$ |